Cloud-Connected Energy Meter with Real-Time Monitoring Using ESP32

Mihit Manoj, P Yathul Anish, Rahul Krishna
Department of Electrical and Electronics Engineering
Amrita Vishwa Vidyapeetham
Kollam, India
mihit.manojm@gmail.com

***Abstract*—This paper presents the design and development of a cloud-connected energy monitoring system using the ESP32 microcontroller and the Adafruit INA219 high-side current sensor. The system measures voltage, current, power, and cumulative energy consumption in real time and transmits these values to the Adafruit IO cloud platform using the MQTT protocol. A custom dashboard enables live visualization of electrical parameters and provides remote control of the connected load. The prototype demonstrates a low-cost, reliable solution suitable for smart metering applications, IoT education, and remote instrumentation. While the current implementation operates without deep-sleep power optimization, the architecture supports future enhancements including ESP32 Ultra-Low-Power (ULP) wake mechanisms, sensor power-gating, and autonomous hardware-based wake triggers to improve system efficiency and scalability.**

***Index Terms*—IoT, ESP32, INA219, Smart Energy Meter, Real-Time Monitoring, Cloud Computing, MQTT, Adafruit IO, Power Measurement, Energy Monitoring, Wireless Sensor Systems, Embedded Systems.**

I. Introduction

The increasing demand for intelligent energy management systems has accelerated the adoption of Internet of Things (IoT)-based solutions for monitoring and controlling electrical loads. Traditional energy meters provide only cumulative consumption data and lack real-time visibility, remote accessibility, and integration capabilities with modern automation platforms. These limitations reduce their effectiveness in contexts where continuous monitoring and adaptive control are required, such as smart homes, laboratories, and industrial environments.

Advancements in low-cost microcontrollers equipped with wireless communication, such as the ESP32, have enabled the development of compact and scalable smart metering systems. Coupled with precision sensors like the Adafruit INA219, it becomes possible to measure electrical parameters—including voltage, current, and power—with high accuracy through digital communication interfaces. Cloud platforms such as Adafruit IO further extend system functionality by providing real-time data visualization, remote device interaction, and persistent storage.

This project presents the design and implementation of a cloud-connected energy meter that leverages the ESP32 microcontroller and INA219 sensor to measure and publish electrical parameters to the Adafruit IO cloud platform using MQTT. The system includes a remote load-control feature through the cloud dashboard and maintains cumulative energy consumption across resets using non-volatile memory. The proposed prototype serves as a flexible and low-cost solution suitable for educational environments, IoT experimentation, and early-stage smart metering research. Future enhancements such as deep sleep, ULP wake-up logic, and power gating are identified to optimize system efficiency and extend operational lifetime.

Identify applicable funding agency here. If none, delete this.

II. Literature Review

Energy monitoring and metering have become an active area of research and development with the advent of low-cost microcontrollers, digital sensors and ubiquitous networking. Early works focused on remote acquisition using GSM and Wi-Fi modules to relay meter readings to centralized servers; recent research emphasizes low-power edge nodes, accurate sensing, and cloud integration for visualization and analytics. The literature can be grouped into (a) sensing and instrumentation, (b) embedded/firmware strategies, (c) low-power operation and wake strategies, and (d) cloud/platform integration — each area informs design trade-offs for the present project.

Sensing and instrumentation. Accurate current and power measurement at low cost is commonly achieved using either analogue front-ends (shunt + amplifier, CTs, op-amps) or integrated digital sensors such as INA219/INA226. Several studies show that instrumentation amplifiers or specialized ADCs produce better resolution for small currents but at the expense of additional components and calibration effort [1]. The INA219 is frequently used in prototypes because it provides high-side current sensing, on-board calibration, and direct I²C access — simplifying firmware while providing sufficient accuracy for many DC applications [2]. However, papers and application notes also caution that sensor selection, shunt sizing and proper calibration strongly affect absolute accuracy and dynamic response (especially for motor inrush currents), so system design must account for these limits.

Embedded design and firmware strategies. Choice of microcontroller and firmware architecture governs responsiveness and feature set. The ESP32 family is widely adopted in recent

literature because it combines dual-core processing, robust Wi-Fi stacks, and low-power features (deep sleep and ULP) in a single chip [3]. Practical implementations typically use Arduino or ESP-IDF frameworks and off-the-shelf libraries (e.g., Adafruit INA219) to accelerate development. Several project reports highlight common pitfalls: blocking network calls during MQTT connect, excessive publish rates causing throttling on cloud services, and timing issues when toggling peripherals around sleep/wake cycles. Best practice emerging from field reports is to separate sensing, network and persistence tasks and to rate-limit publishes so cloud quotas are not exceeded.

Low-power operation and wake strategies. For battery-powered or energy-constrained deployments, deep sleep and ULP strategies have been proposed to dramatically reduce average consumption. The ESP32's ULP coprocessor can run very low-power routines (ADC checks, GPIO monitoring) while the CPU sleeps — enabling event-driven wakeups without keeping the main core active [4]. Other approaches use external analog comparators or hall/current sensors to produce a hardware wake signal (EXT0/EXT1) — this is attractive when the main controller must remain off until a threshold event (e.g., motor start) occurs [5]. The literature emphasizes trade-offs: ULP programming is more complex and ADC accuracy in RTC domain can be limited; external hardware adds components but provides reliable, low-latency wake signals and often lower quiescent draw if designed correctly.

Cloud and platform considerations. Many implementations use MQTT-based cloud platforms (Adafruit IO, ThingsBoard, AWS IoT) for telemetry, visualization and remote actuation. Adafruit IO is popular in academic prototypes for ease of use and dashboarding capabilities, but its free-tier rate limits and simplistic feed model can lead to throttling if multiple feeds are published frequently [6]. Researchers recommend aggregating data into compact payloads (e.g., JSON packets) or designing server-side parsers to avoid hitting publish quotas. Security, authentication and reliable reconnection strategies are recurring themes in the literature because intermittent Wi-Fi and network outages are common in practical deployments.

Gaps and motivations for this work. Review of recent reports and practical guides reveals common gaps: (1) many prototypes neglect power optimization (deep sleep / ULP) and hence are unsuitable for long-term battery operation; (2) handling motor inrush and rapid state changes (ON/OFF) reliably while minimizing wakeups remains challenging; (3) cloud throttling and feed management is often an afterthought, leading to unreliable dashboards under realistic use. This project targets these gaps by (a) using INA219 for accurate, straightforward DC sensing, (b) planning for ULP/hardware wake approaches in future work to support automatic wake on motor events, and (c) applying payload aggregation and rate-limiting to avoid cloud throttling while preserving real-time responsiveness.

III. System Design

A. Hardware Architecture

The hardware architecture of the proposed system is centered on the ESP32 microcontroller, which performs sensing, computation, and cloud communication. To obtain electrical measurements, the ESP32 interfaces with the Adafruit INA219 high-side current sensor over the I²C protocol. The sensor's SDA and SCL pins are mapped to GPIO21 and GPIO22 of the ESP32 respectively, while the INA219 is powered through the ESP32's 3.3-V supply rail. The measurement terminals of the INA219, labeled VIN+ and VIN, are placed in series with the positive supply line of the DC load. This configuration enables the sensor to measure the bus voltage at VIN and the shunt voltage between VIN+ and VIN, from which it internally computes current and instantaneous power. A common ground reference is shared by the ESP32, sensor module, and load to ensure measurement stability and noise-free digital communication.

The electrical load used for demonstration, such as a small DC motor or LED, may be driven directly through the ESP32 in the current version of the project or externally controlled through a manual switch. The system also incorporates a flyback protection diode across inductive loads to protect the MOSFET and ESP32 circuitry. Although the present version does not employ hardware-triggered wake-up, the architecture allows future integration of comparators or op-amp circuits connected to an RTC-capable GPIO for event-based wake conditions.

B. Firmware Architecture

The firmware running on the ESP32 is organized around periodic sensing tasks, MQTT communication routines, and non-volatile storage operations. Using the INA219 library, the ESP32 periodically polls the sensor to retrieve bus voltage, shunt voltage, current, and power measurements. These values are subsequently processed and converted into standard SI units. Energy accumulation is performed by integrating instantaneous power over elapsed time to obtain milli-watt-hour (mWh) consumption, which is then stored using the Preferences API. This ensures that cumulative energy values are preserved across resets and power cycles.

Cloud connectivity is managed through the Adafruit IO Wi-Fi client library, which uses MQTT for publishing and subscribing to data streams. Sensor readings are uploaded to dedicated cloud feeds at regular intervals—voltage, current, and power every ten seconds, and energy every sixty seconds—to maintain compliance with platform rate limits. The firmware also subscribes to a motor-control feed, enabling remote actuation of the load. Upon receiving a command, the ESP32 updates its output pin and immediately publishes a new set of sensor readings to reflect the changed state. The program includes routines for connection verification, graceful handling of disconnections, and publication rate limiting to ensure robust long-term operation.

C. Cloud Integration and Dashboard Design

Adafruit IO serves as the cloud backend for visualization and remote interaction. The system publishes numerical values to four separate feeds corresponding to voltage, current, power, and cumulative energy. The dashboard displays these values using gauges, numerical widgets, and time-series plots. A toggle switch on the dashboard is linked to the motor-control feed, permitting users to remotely activate or deactivate the connected load. The lightweight MQTT interface allows efficient communication with minimal overhead, making the system suitable even for constrained network environments.

To reduce the risk of MQTT throttling, the firmware manages the frequency of publications carefully and avoids redundant updates. The feed-based approach also makes the system easily scalable, allowing additional sensors or features to be added through new feeds without altering the basic architecture.

D. Timing, Sampling, and Performance Considerations

The system's performance is governed by its sampling and publication intervals. The sensing task retrieves INA219 measurements every ten seconds, a rate that balances responsiveness with network efficiency. Energy accumulation, however, is computed continuously based on the sensor's instantaneous power output and the computed time delta between successive samples. Energy is published every sixty seconds to reduce traffic and maintain adherence to platform rate restrictions.

While the current implementation operates continuously without entering low-power modes, the design has been developed with future enhancements in mind. The ESP32 supports deep sleep and ULP coprocessor features, which can dramatically reduce power consumption during idle periods. Future iterations may implement ULP-based periodic sampling, hardware-triggered wake-up based on motor activity, and power-gating of the INA219 sensor to minimize quiescent current draw.

E. Design Choices and Expandability

The design prioritizes modularity and clarity, selecting digital measurement components (INA219) and a cloud platform (Adafruit IO) that simplify both hardware implementation and software development. The use of MQTT-based communication allows easy adaptation to other platforms such as AWS IoT, ThingsBoard, or custom MQTT brokers. Additionally, the system's architecture supports both DC load monitoring and future extension to AC sensing through the integration of voltage transformer modules and current transformers.

Planned enhancements include combining multiple sensor values into a single JSON payload to further reduce MQTT publish counts, implementing hardware wake logic for automatic system activation when the load is turned on, and adding local storage options for offline logging. These improvements aim to transition the current prototype into a fully optimized, low-power smart metering node.

IV. Methodology

A. Measurement Principle

The primary parameters monitored in this system are voltage, current, and instantaneous power. The INA219 current sensor employs a high-side shunt resistor to measure the small voltage drop across the load supply line. By applying Ohm's law, the sensor internally calculates the current flowing through the load. The bus voltage is measured directly at the output terminal VIN, enabling accurate determination of real-time power consumption as the product of voltage and current. The sensor communicates the processed data digitally over the I²C interface, minimizing external signal conditioning requirements and ensuring high measurement accuracy.

Energy consumption is computed using numerical time integration. Instantaneous power values sampled periodically are multiplied by the elapsed sampling interval and accumulated to generate milli-watt-hour (mWh) readings. This computation is executed continuously to ensure proper energy tracking during the device's active monitoring period.

B. Embedded Firmware Workflow

The firmware executed on the ESP32 follows a structured routine of measurement, processing, and cloud communication. Upon initialization, the Wi-Fi module connects to the configured network and establishes an MQTT session with the Adafruit IO server. The ESP32 retrieves the previously stored energy values from non-volatile memory to ensure continuity of reporting across resets.

The INA219 sensor is polled at a periodic interval of ten seconds. The raw data is converted into readable electrical parameters, which are subsequently uploaded to the respective cloud feeds. The MQTT architecture allows the ESP32 to efficiently publish telemetry data while simultaneously subscribing to incoming control messages from the dashboard. Incoming MQTT messages trigger an interrupt-driven callback function that updates the state of the load and forces an immediate data refresh to the cloud, ensuring user commands reflect instantly in the monitored output values.

C. Cloud Communication and MQTT Handling

The MQTT protocol enables lightweight and reliable communication between the ESP32 and the Adafruit IO cloud platform. Each measured parameter is assigned to an individual feed, allowing organized and scalable visual representation. The firmware incorporates adaptive timing to avoid exceeding the publishing limits imposed by the platform. Energy data is published every sixty seconds, while other parameters are transmitted at higher frequency due to their dynamic nature.

The dashboard is configured with graphical widgets such as gauges and line plots to provide intuitive real-time visualization to the user. A toggle switch widget allows remote load control by sending a digital command to the motor-control feed. The ESP32 acknowledges and executes the command promptly, completing the bidirectional IoT communication loop.

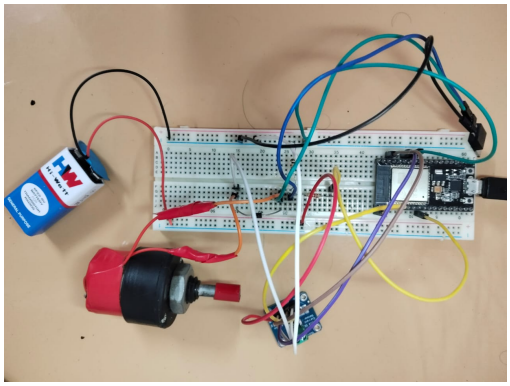


Fig. 1. Hardware setup

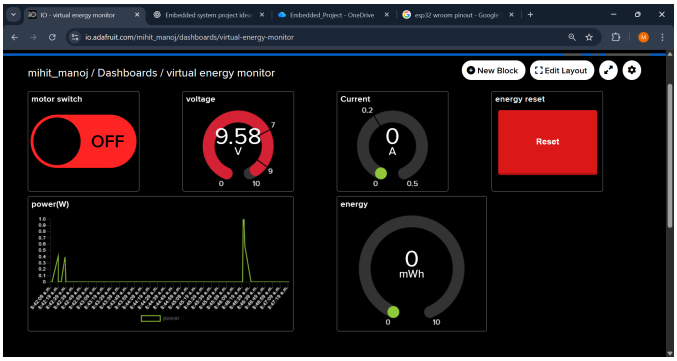


Fig. 2. Dashboard when motor is OFF

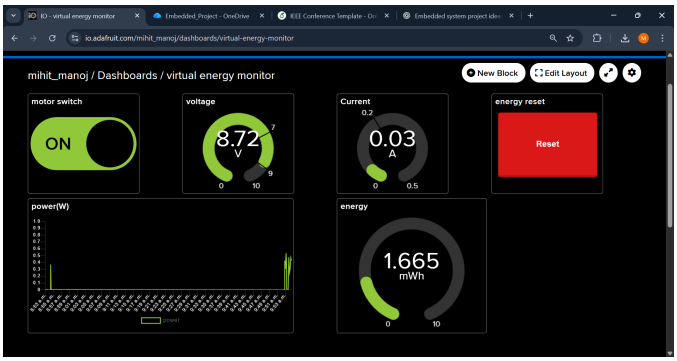


Fig. 3. Dashboard when motor is ON

D. Persistent Data Storage

To ensure reliability in real-world deployment environments, the system stores cumulative energy data in the on-board flash memory using the ESP32 Preferences API. This prevents data loss caused by power interruption or reset events. The stored value is loaded during initialization, updated continuously during runtime, and re-written to non-volatile memory at controlled intervals to minimize memory wear. This method ensures accurate lifelong energy accumulation without requiring continuous cloud connectivity.

E. Development and Testing Procedure

The system was initially validated through bench testing using a controlled DC motor load powered from a regulated supply. Calibration steps included verifying stable I²C communication, confirming expected sensor output values under known mechanical load variations, and ensuring measurement accuracy by comparing readings with a commercial multimeter. Cloud connectivity, MQTT message reliability, and dashboard responsiveness were evaluated through prolonged runtime tests and repeated remote switching operations.

During testing, considerations were made to evaluate network delay, sensor noise, and transient current spikes associated with motor startup. Error-handling routines were introduced into the firmware to maintain stable operation in the presence of temporary communication failures. This testing methodology ensures suitability of the system for continuous deployment in dynamic IoT environments.

V. Results

The proposed cloud-connected energy monitoring system was successfully developed and tested in a practical setup. A DC motor was used as the load, and its operation demonstrated the correct functioning of the measurement, communication, and control subsystems. Figure 1 shows the hardware prototype consisting of the ESP32, INA219 current sensor, and the connected motor. All components were assembled on a breadboard to validate rapid prototyping and real-time behavior. During operation, the ESP32 continuously measured voltage, current, and instantaneous power through the INA219 and uploaded the data to Adafruit IO. The dashboard accurately displayed real-time variations in the electrical parameters through numerical indicators and live graphs. Figures 2 and 3 illustrate the dashboard behavior when the motor was remotely switched ON and OFF respectively. When the motor was activated, the measured current and power increased significantly, reflecting the mechanical load placed on the system. Conversely, both values approached near-zero when the motor was turned OFF, confirming correct measurement and control functionality. The trend plots recorded over a continuous monitoring interval are shown. Distinct transitions are visible at motor switching events, demonstrating accurate tracking of dynamic load changes. The energy consumption feed also showed continuous incremental growth in milliwatt-hours (mWh) during ON periods, validating the correct functioning of the integration algorithm and the non-volatile storage mechanism.

Remote load switching through the dashboard performed reliably, with each toggle triggering an immediate sensor update and visual confirmation. No data throttling was observed during testing with the configured publishing rates. Additionally, the system remained stable over an extended runtime, demonstrating robust connectivity and cloud synchronization.

Overall, the results confirm that the system meets the objectives of real-time energy monitoring, remote control capability, and reliable cloud-based data visualization. The prototype serves as a functional baseline for future enhance-

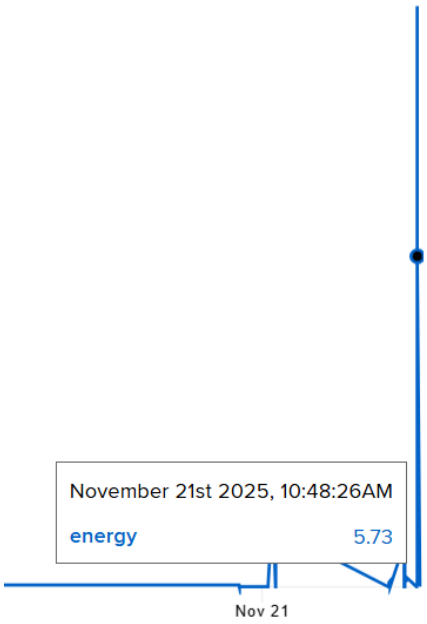


Fig. 4. The energy graph for a motor load

ments targeting power-optimized operation and autonomous wake mechanisms.

VI. Discussion

The evaluation of the proposed cloud-connected smart energy meter demonstrates that the system is capable of reliably monitoring and controlling a DC load through IoT infrastructure. The measurements obtained from the INA219 sensor exhibited stable and consistent behavior under varying operating conditions, indicating accurate sensing of both steady-state and transient electrical parameters. This validates the choice of a digital high-side current measurement technique, which eliminates ground interference and ensures safer integration with the load.

The cloud publishing mechanism using MQTT proved effective in delivering real-time updates to the remote dashboard. The observed responsiveness during motor switching events confirms that the firmware and dashboard are synchronized, enabling users to remotely control the load with minimal latency. The incremental change in measured current and power during the ON state, along with the corresponding reduction in the OFF state, aligns with expected electrical behavior and further substantiates the correct functioning of the sensing and control pipeline.

The visual plots on the Adafruit IO dashboard clearly reflect the dynamic transitions in energy usage. This supports the integration method for cumulative energy estimation using numerical power-time integration stored in non-volatile memory. Since energy readings were preserved across resets, the design demonstrates suitability for prolonged or discontinuous monitoring applications where power interruptions may occur.

However, the current version of the system operates continuously without implementing any power-saving strategies. As a result, the ESP32 and INA219 remain powered throughout the monitoring period, which would lead to significant battery drain in standalone deployments. Communication frequency, while controlled to avoid MQTT throttling, still contributes to unnecessary power usage when no load is active. This highlights an important improvement area for real-world adoption. Additionally, the system presently uses manual dashboard control for switching the load; autonomy could be enhanced by incorporating hardware-based sensing and wake-up techniques to detect when the load is physically enabled.

Overall, the system demonstrates strong feasibility and reliability in its present form. The findings indicate that the proposed hardware-cloud architecture can be effectively extended with low-power modes and autonomous activation capabilities to achieve a fully optimized smart metering solution suitable for IoT-driven energy management environments.

VII. Conclusion

A cloud-connected energy monitoring system using ESP32 and INA219 has been successfully designed and implemented. The prototype demonstrates accurate real-time measurement of voltage, current, and power, along with cumulative energy tracking and reliable remote control through the Adafruit IO platform. Persistent storage ensures that energy data is preserved across resets, confirming suitability for long-term smart metering applications.

Future enhancements will focus on reducing power consumption through deep-sleep operation, ULP-based wake-up mechanisms, and power gating of the sensor. Additional improvements such as hardware-triggered activation and optimized cloud communication will further increase efficiency and autonomy, enabling the system to evolve into a fully optimized and deployable smart energy meter.

References

[1] Espressif Systems, ESP32 Series Datasheet, 2023. [Online]. Available: https://www.espressif.com/
[2] Adafruit Industries, INA219 High-Side Current Sensor Breakout – Product Guide, 2024. [Online]. Available: https://www.adafruit.com/
[3] Texas Instruments, INA219 Zero-Drift, Bi-Directional Current/Power Monitor Datasheet, 2015.
[4] Adafruit IO Documentation, "Adafruit IO MQTT API Reference," 2024. [Online]. Available: https://io.adafruit.com/
[5] Espressif Systems, ESP-IDF Programming Guide – Deep Sleep and ULP Coprocessor, 2024. [Online]. Available: https://docs.espressif.com/
[6] Adafruit Industries, Adafruit INA219 Arduino Library — Documentation, 2024.
[7] J. A. Stankovic, "Research directions for the Internet of Things," IEEE Internet of Things Journal, vol. 1, no. 1, pp. 3–9, Feb. 2014.
[8] G. E. Santillán, I. K. Yousif and R. A. Rashid, "IoT-based Smart Energy Meter Monitoring System," International Journal of Electrical and Computer Engineering, vol. 10, no. 2, pp. 1569–1577, 2020.
[9] P. S. Nayak and S. Mohapatra, "Design and Implementation of Wi-Fi Based Smart Energy Meter," IEEE Conference on Power, Control, Signals and Instrumentation Engineering, pp. 148–153, 2019.
[10] K. N. Qureshi, A. H. Mir, "IoT-Driven Smart Grid: Cloud–Based Real-Time Energy Monitoring," IEEE Access, vol. 8, pp. 201900-201910, 2020.
[11] H. El-Sayed et al., "Edge Computing Architecture for Smart Grid Applications," IEEE Communications Magazine, vol. 57, no. 9, pp. 72–78, 2019.